



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0416-Operating Systems for Resource Management

Technical Presentation

Guided by :

Dr. Vasavi J

Presented by :

A Mohammad Naheed
(192525336)

B.TECH -AIML



ENGINEER TO EXCEL



Scenario 1

Suppose an Operating System is used in a banking system handling thousands of transactions simultaneously.

presentation explaining : Types of OS suitable for this scenario OS structure used Role of system calls in transaction processing



ENGINEER TO EXCEL



Q 1: OS Types for High-Volume Banking Transactions

•Distributed OS (Core Banking Backend)

→ Multiple servers handle transactions (load split across nodes)

→ Example: One server handles account validation, another handles balance update

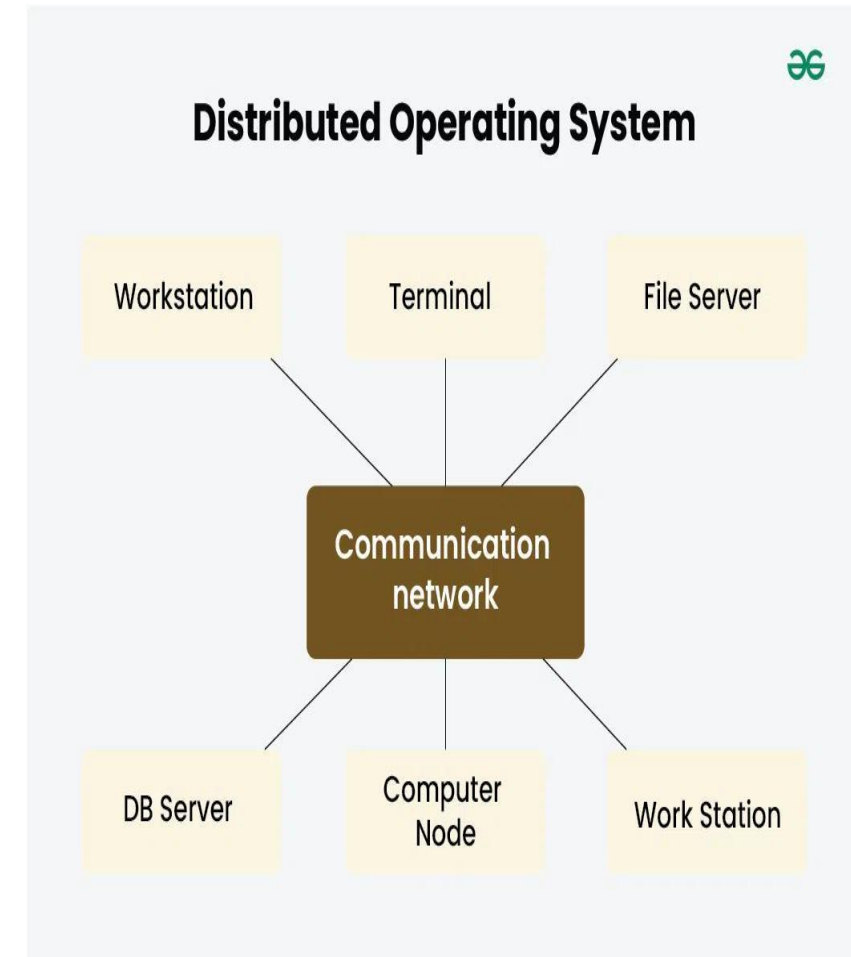
•Multiprocessing OS (Database Servers)

→ Parallel execution of transactions using multiple CPUs

→ Prevents delay during peak hours (salary day, festivals)

•Real-Time OS (ATM / Payment Gateways)

→ Ensures **instant response** (e.g., cash withdrawal must not delay)



Q 2: OS Structure Used



- **Actual Structure Used: 3-Tier + Client-Server**
- **Presentation Layer (Clients)**
 - ATM, Mobile App, Internet Banking
- **Application Layer (Servers)**
 - Validates transaction (PIN check, fraud detection)
- **Database Layer**
 - Stores account balance, transaction history
- **OS Role in Structure:**
 - Handles **process scheduling** for thousands of requests
 - Ensures **isolation** (one failed transaction doesn't affect others)
- Provides **inter-process communication (IPC)** between layers



ENGINEER TO EXCEL



Q 3:

Role of System Calls in Transaction Processing

- **System Calls Used:**
- **read() / write()**
 - Read account details from database
 - Update balance after withdrawal
- **fork() / process creation**
 - Each transaction handled as a **separate process/thread**
- **open() / close()**
 - Access secure banking database
- **send() / receive() (network calls)**
 - Communicate between ATM → bank server
- **Security-related calls**
 - Verify user authentication & permissions
- **Example: ATM Withdrawal Transaction**
 - User inserts card → enters PIN
 - Application sends request to OS



ENGINEER TO EXCEL



Scenario 2

Consider a multi-user Linux-based server environment supporting file sharing and device access.

presentation covering : OS functions involved in resource management File and device

system calls Interaction between user processes and kernel

Q 1:

OS Functions in Resource Management

- **CPU Scheduling (CFS – Completely Fair Scheduler)**

- Ensures fair CPU time for all users' processes

- Prevents one user from hogging the system

- **Memory Management**

- Each user process gets **isolated virtual memory**

- Uses paging & swap to avoid crashes under load

- **File System Management**

- Controls access to shared files using permissions (rwx)

- Prevents unauthorized access

Device Management

- Devices treated as files (/dev/)

- OS manages access via drivers



ENGINEER TO EXCEL

Q 2:

File & Device System Calls in Real Use



File System Calls (Used in File Sharing):

`open()` → Open shared file (e.g., `project.txt`)

`read()` → Read file contents

`write()` → Modify/update file

`close()` → Release file after use

👉 Example: Two users editing same file → OS ensures **controlled access (locking/permissions)**

Device System Calls (Devices as Files):

`open("/dev/printer")` → Access printer

`read()` / `write()` → Send data to device

`ioctl()` → Control device behaviour

👉 Example: Multiple users printing → OS queues requests (spooling)



ENGINEER TO EXCEL

Q 3:

Interaction Between User Processes & Kernel



- **How Interaction Happens:**

- User runs a program (e.g., file editor via SSH)
- Program runs in **User Mode**
- Needs file/device → makes **system call**
- Control switches to **Kernel Mode**
- Kernel performs operation safely
- Returns result to user process

- **Key Mechanisms:**

- **System Call Interface (SCI)** → Gateway to kernel
- **Context Switching** → Switch between processes/users
- **Access Control** → Kernel checks permissions before action



ENGINEER TO EXCEL



Scenario 3

Assume a real-time embedded system used in traffic signal control.

**presentation explaining : Type of OS used and justification OS structure and components
System calls used for process control and timing**

Q 1:

Type of OS Used & Justification



- **Type of OS Used: Real-Time Operating System (RTOS)**
Examples: FreeRTOS, VxWorks
- **Justification:**
- **Deterministic timing** → Signals must change at precise intervals
- **Deadline-based execution** → Missing a deadline can cause accidents
- **Priority scheduling** → Emergency vehicle signal gets highest priority
- **Low latency response** → Immediate reaction to sensor input
- 🙌 General-purpose OS (like Linux desktop) is **not suitable** due to unpredictable delays

Q 2:

OS Structure & Components in Traffic System

- **Key Components:**
- **Task Scheduler**
 - Decides which task runs (light control, sensor reading)
- **Interrupt Handlers**
 - Handle real-time events (vehicle detected, pedestrian button)
- **Timers**
 - Maintain exact signal timing (e.g., 30s green)
- **Device Drivers**
 - Control LEDs (signals), sensors, buzzers
- **Inter-Process Communication (IPC)**
 - Tasks share data (sensor → control logic)



Q 3: System Calls for Process Control & Timing



- **Process Control System Calls:**

- `task_create()` / `fork()`
→ Create tasks (e.g., signal control task)
- `task_suspend()` / `resume()`
→ Pause normal flow for emergency override

- **Timing System Calls:**

- `sleep()` / `delay()`
→ Maintain fixed light durations (e.g., 60 sec red)
- `set_timer()`
→ Configure countdown timers
- `get_time()`
→ Track elapsed time for synchronization

- **Interrupt-Based Calls:**

- Triggered when:
 - Vehicle detected
 - Pedestrian button pressed



Scenario 4

A company uses a distributed computing system across multiple locations.

- **presentation discussing :Type of OS and its characteristics Architecture of distributed OS System calls used for communication and coordination**



Q 1: Type of OS & Its Characteristics



- **Type of OS Used: Distributed Operating System**

Examples: Amoeba Distributed Operating System, Plan 9 from Bell Labs, modern setups built on Linux clusters

- **Key Characteristics (in this scenario):**

- **Transparency**

→ Users access resources as if on one system (no need-to-know server location)

- **Scalability**

→ New servers can be added as company grows

- **Fault Tolerance**

→ If one node fails, others continue (important for business continuity)

- **Resource Sharing**

→ Files, printers, databases shared across locations

Q 2:

Architecture of Distributed OS

- **Components:**
- **Client Nodes (User Systems)**
 - Employees accessing applications
- **Application Servers**
 - Process business logic (e.g., payroll, inventory)
- **Database Servers**
 - Store centralized or replicated data
- **Network Layer**
 - Connects all nodes (LAN/WAN)
- **Distributed OS Role:**
- Manages **communication between nodes**
- Handles **load balancing** (distributes tasks)
- Ensures **data consistency** across locations



ENGINEER TO EXCEL



Q 3:

System Calls for Communication & Coordination

- **Communication System Calls:**

- `socket()` → Create communication endpoint
- `connect()` → Establish connection to remote server
- `send()` / `recv()` → Transfer data between nodes
- 👉 Used when client sends request to remote server
-

- **Coordination & Synchronization Calls:**

- `fork()` / process creation → Parallel processing across nodes
- `wait()` → Synchronize dependent processes
- `semaphore()` / locks → Prevent data conflicts (e.g., two users editing same record)



ENGINEER TO EXCEL



Scenario 5

Consider an OS managing process scheduling, memory allocation, and I/O operations in a modern computer

- **presentation including : Key OS functions and their importance
Interaction between OS components Examples of system calls
for each function**

Q 1: Key OS Functions & Their Importance

1. Process Scheduling

→ Decides which process gets CPU

→ Uses algorithms (Round Robin, Priority)

Importance: Smooth multitasking (no app freezing)

2. Memory Management

→ Allocates RAM to processes

→ Uses **virtual memory** when RAM is full

Importance: Prevents crashes, ensures isolation

3. I/O Management

→ Handles input/output devices (keyboard, disk, printer)

→ Uses buffering & caching

Importance: Fast and efficient device usage

Q 2 : Interaction Between OS Components

- **How Components Work Together:**
- **Process Scheduler ↔ Memory Manager**
→ Scheduler selects process → Memory must already be allocated
- **Process ↔ I/O Manager**
→ Running process requests file/device → I/O manager handles it
- **Memory Manager ↔ I/O Manager**
→ Data read from disk → loaded into memory
- **Kernel Role**
→ Central controller coordinating all components
- 🙌 **Example Flow:**
User opens file → process created → memory allocated → disk read → data displayed

Q 3: System Calls for Each Function

- **Process Scheduling Calls**

- `fork()` → Create new process
- `exec()` → Load program into process
- `wait()` → Synchronize processes
- 🙌 Example: Opening browser creates multiple processes

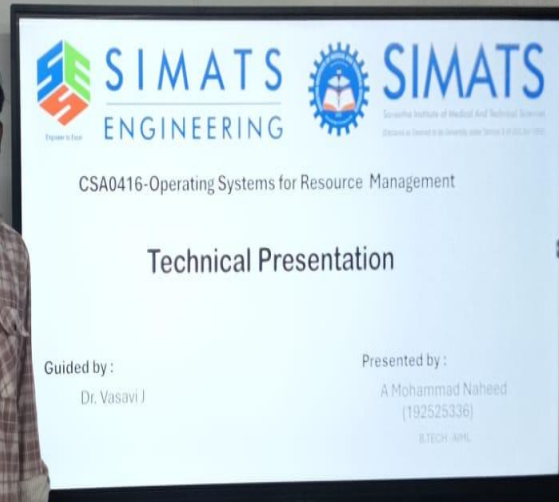
- **Memory Management Calls**

- `malloc()` / `brk()` → Allocate memory
- `mmap()` → Map files into memory
- `free()` → Release memory
- 🙌 Example: App requests memory to load data

I/O System Calls

`open()` → Open file/device
`read()` / `write()` → Perform I/O
`close()` → Release resource
`ioctl()` → Control device

🙌 Example: Reading a file from disk



GPS Map Camera

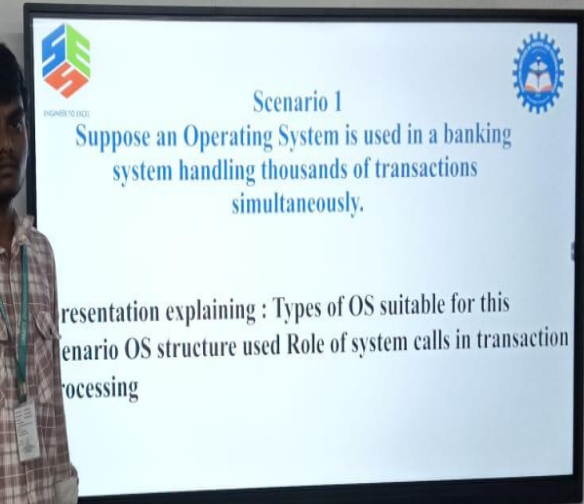


Kuthambakkam, Tamil Nadu, India

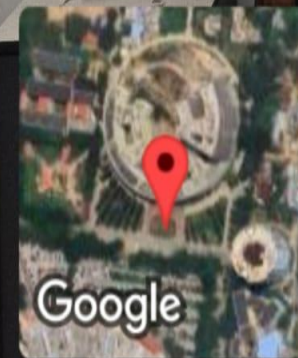


22g8+9hp, Kuthambakkam, Tamil Nadu 602105, India
Lat 13.026029° Long 80.016488°

Wednesday, 29/04/2026 03:35 PM GMT +05:30



GPS Map Camera



Kuthambakkam, Tamil Nadu, India



22g8+9hp, Kuthambakkam, Tamil Nadu 602105, India
Lat 13.026021° Long 80.016493°

Wednesday, 29/04/2026 03:35 PM GMT +05:30

Thank
you!